18. Инженеринг на изискванията като част от системния и софтуерен инженеринг. Представяне на техники, прилагани на отделните етапи от инженеринга на софтуерните изисквания. Основни методи за моделиране.

1) Цел и задачи на инженеринга на софтуерните изисквания.

Цели и задачи на инжинеринга на изискванията:

- Да създаде и поддържа документ със системните изисквания.
- Да идентифицира целта на софутерната система и контекстите, в които ще бъде използвана.
- Да играе ролята на мост между реалните нужди на потребителите, клиентите и други, засенати от системата, и възможностите, които се предлагат от софтуерните технологии.
- Да дефинира обхвата на продуктите, които се разработват.
- Да установи и специфицира прецизно какво трябва да прави софтуера без да описва как го прави.
- Да разбере и определи проблема, който трябва да се реши чрез разработката на софтуера.
- Да разбере какво се изисква от софтуера.
- Да комуникира разбирането за това какво се изисква с всички заинтересовани лица.
- Да осигури начини за контрол, които да осигурят, че финалната система задоволява изискванията (включително управлението на ефекта от промените).
- Да определи целите, функциите и ограниченията към софтуерните системи, както и връзката между тези фактори.
- Създаването на прецизна спецификация на поведението на софтуера и еволюцията му.
- Да определи какво трябва да прави софтуера за да добави стойност за заинтересованите лица – функционални изисквания.
- Да определи какво трябва а представлява софтуера – нефункционални изисквания.
- Да определи какви са ограниченията при имплементацията.

2) Параметри на процеса

Класификация на изискванията:

- функционални и нефукнкционални
- бизнес и потребителски
- спрямо устойчивост (stability)
    - стабилни - не се променят по време на разработката, свързани са със същитната на системата или приложната й област.
    - променливи - изисквания, които се изменят с времето или употребата на системата. Характерни са за инстанция на системата в определена среда и за определен клиент.
- съгласно приотиризация
    - задължителни
    - препорачителни
    - възможни - могат да бъдат елиминирани
- класификация на Pfleeger

    - функционални
    - качествени - нефункционални
    - ограничения към дизайна - решение на дизайна
    - ограничения към процеса - ограничение върху техниките или ресурсите, които могат да се използват за построяването на системата.
- Класификация:
    - Променливи/Мутиращи (Mutable) – изиквания, променящи се поради средата на системата.
    - Появяващи се (Emergent) – изисквания, които се появят в процеса на по-добре разбиране на системата.
    - Последващи (Consequential) – изисквания, които прозтичат от представянето на системата.
    - Съвместими (Compatibility) – изисквания, зависещи от други системи и организационни процеси.

Видове изисквания:

- функционални - описват желаното поведение под формата на сървиси, задачи или функции, които системата трябва да изпълнява. Дефинират границите на решението на проблема.
- нефункционални - описват ограничения върху системата или разработката. Описват изискваните свойства или качества на системата. Има 2 типа:
    - Observable at runtime
    - non observable at runtime
- Изисквания на приложната област - произлизащи от приложната област на системата, които отразяват спецификите ѝ.
- бизнес изисквания - представят целите от високо ниво на организацията или клиента, който поръчва системата. Описват защо организацията имплементира системата и целите, които желае да бъдат постигнати. Представляват бизнес deliverable, които осигуряват стойност. Формулират се от перспективата и на езика на бизнеса или потребителя.
- потребителски изисквания - описват целите или задачите, които потребителите трябва да могат да изпълняват с продукта - какво потребителят трябва да може да прави със системата.
- продуктови изисквания - изискванията от гледна точка на дефинирания продукт, който е един от начините за задоволяването на бизнес изискванията. Осигуряват стойност единствено, ако отговарят на истинските бизнес изисквания.
- системни изисквания - описват изискванията от високо ниво за продукт, съдържащ няколко подсистеми. Структуриран документ с детайлно описание на системните функции, услуги и оперативни ограничения.
- процесни изисквания - ограничения върху процеса на разработка. Обикновено се включват при големи организации с установени стандарти и практики.
- външни изисквания - изисквания, които могат да са наложени върху продукта и процеса, и са получени от средата, в която се разработва системата.

Видове нефункционални изисквания:

- класификация според IEEE
    - performance
    - interface
    - operational
    - resource
    - verification
    - acceptance

  - documentation
  - security
  - portability
  - quality
  - reliability
  - maintainability
  - safety
- класификация според Sommerville
  - подуктови - Изисквания, които определят поведението на продукта, като време за изпълнение, надежност и др. (delivery, implementational, standards).
  - процесни - Изисквания, които са следствие на практиката и процедурите в дадена организация, като прилаганите стандарти при процеса на разработване, изисквания към имплементцията и др. (usability, reliability, safety, efficiency(performance, space), portability, capacity).
  - външни - Изисквания породени от външни за системата и процеса на разработване фактори, като изисквания за оперативна съвместимост и нормативни (законодателни) изисквания и др. (legal, economic, interoperability(privacy, safety), ethical)

Заинтересовани лица (stakeholders):

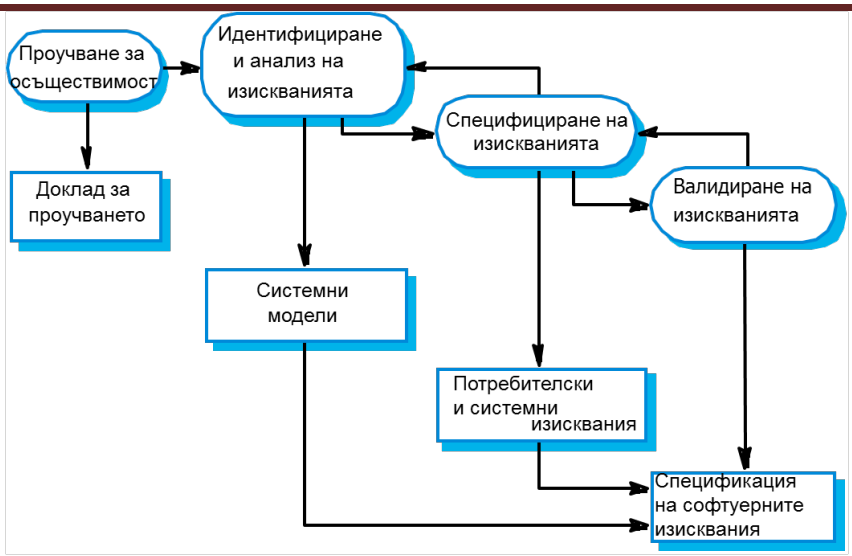
- клиенти, които плащат за разработката
- клиенти, които купуват софтуера след като е разработен
- потребители
- експерти в приложната област, запознати с проблема, който софтуерът трябва да автоматизира
- адвокати и одитори
- софтуерни инженери и др. технологични експерти
- хора, проучили пазара, за да опраделят бъдещите тенденции и потенциалните нужди на клиентите.

Потребители

Те са различни и трябва да се обединят в няколко категории. Всяка категория има свой изисквания, които трябва да бъдат взети впредвид.

3) Етапи (дейности) на инженеринга на изискванията.

18. Инженеринг на изискванията като част от системния и софтуерен инженеринг.



Етапите при инженеринга на изискванията са:

- проучване за осъществимост (feasibility study) – решават дали предложената система си заслужава. Събират информация относно дали е възможно да се постигнат целите на проекта, дали системата би допринесла за целите на организацията, дали системата може да бъде интергирана с останалите системи в организацията и дали може да се постигне съгласие относно контекста на работата. Проучването за осъществимост завършва с изготвянето на доклад за осъществимост (feasibility report), който трябва да дава препоръка дали разработката да продължи.
- извличане на изисквания - състои се от разучаване на проблемната област и извличането на изискванията. Основните задачи са да се идентифицират заинтересованите лица, откриването и разрешаването на конфликти между твърденията на различните лица, идентифициране и постигане на съгласие относно границите на системата. Необходимо е да се постигне разбиране на приложната област, проблема, бизнеса и нуждите и ограниченията на системата от заинтересованите лица. Основната цел на извличането на изискванията е да се определят проблемът, който се решава, и границите на системата, която ще се разработва. В процесът по извличане на изискванията се създава чернова на документа със системните изисквания.
- Анализ на изискванията – при него се включват действия по разбирането на приложната област, събиране на изисквания, класификацията им, решаване на конфликти между тях, приотиризация и проверка на изискванията. По време на анализа могат да бъдат изготвени различни модели на системата. Проверява се дали изискванията са необходими, консистентни, цялостни и реализируеми. Целта на анализа е да откире пробеми, непълноти и неконсистентности в извлечените изисквания.
- дефиниране на изискванията - изготвяне на дефиниции и диаграми на услугите, които системата предоставя и нейните огранчиения.
- спецификация на изискванията - изготвяне на детайлно софтуерно описание, което да осигури база за дизайна и имплементацията. Формализира реултатите от извличането и анализа в документ.
- валидиране на изискванията - осигурява, че изискванията отговаят на желанията на клиента. В този етап се работи с крайния вариант на формулираните изисквания и те се проверяват за пълнота, последователност, съответствие със стандартите, конфликти между изискванията, неясни такива, технически грешки и греши в моделите на системата.
- управление на изисквания - занимава се с осигуряването на ефективен контрол върху системните изисквания и на интегритет на информацията през целия жизнен цикъл на

системата като има в предвид и промените в системата и средата й. Занимава се с управлението на промените в изискванията за системата.

4) Техники за извличане, анализ и валидиране на изискванията.

Техники за извличане и анализ на изисквания:

- Viewpoint-oriented elicitation – Заинтересованите лица представят различни гледни точни на проблема. При този подход се анализират перспективите на различните stakeholder-и. Примери за viewpoint-oriented методи са SADT, CORE.
- Сценарии – представляват описания как системата би била използвана на практика. Включват описания на състоянието на системата преди и след изпълнението на сценария, нормалния поток на събитията, какво би могло да се обърка и как се управлява, други конкурентни дейности.
- Use cases – техника в UML, базирана на сценарии, за идентиициране на участниците в интеракция със системата и самата интеракция. Множество use case-и трябва да описват всички възможни интеракции със системата
- Интервюта – инженерът по изискванията или аналистът дискутира системата с различни заинтересовани лица и изгражда представа за тяхните изисквания. Предимуществата на тази техника са, че се провеждат с различни заинтересовани лица, събира се много информация, а недостатък е, че информацията идва от множество източници.
- FAST (Facilitated Application Specification Technique) – представляват структурирани срещи между разработчиците и клиентите, целящи да определят основните изисквания. Основната идея на тази техника е да запълни дупката между представите на разработчиците за това, което ще разработват, и на клиентите за това, което ще получат.
- Soft System methods – създават неформални модели на системата. Взимат впредвид системата, хората и организацията. Не са подходящи за детайлно извличане на изискванията. Най-популярният от тях е Software Systems Methodology.
- Наблюдение и социален анализ (Observation & social analysis) – аналистът наблюдава процеса на работа на хората, които са бъдещите потребители на системата, и си изгражда представа на това как се върши работата.
- Преизползване на изисквания – включва взимането на изискванията, разработени за една система и използването им в различна система. Спестява врме и усилия, тъй като преизползваните изисквания вече са били анализирани и валидирани в други системи.
- Прототипиране – разработва се прототип на системата и се предоставяна заинтересованите лица. Има 2 типа:
    - Throw-away прототипиране – идеята на прототипът е да помогне при извличането на системните изисквания. Трябва да се разработят единствено най-неясните изсквания.
    - Еволюционно прототипиране – целят да предоставят работеща система на потребителя възможно най-бързо. Първо се разработват най-добре разбраните изиквания.
- Въпросници – информация се събира под формата на проучване. Позволява да се достигнат много хора, но е трудно да се събере контекстуална информация. Трудна е и разработката на въпросниците.
- Брейнсторминг – подходяща при неясни изисквания.
- Storyboards – идентифицират се участниците, какво се случва с тях и как при използването на системата.
- Role playing – позволява на заинтересованите лица да изпитат света на потребителя от неговата перспектива.

Техники за валидиране:

- Преглед/рецензия(review) на исканията – най-често използваната техника за валидиране. Формира се специална група, която да се запознае с изискванията, да ги анализира, да формулира забелязани проблем и да дискутира възможностите за справяне с откритите проблеми. Необходимо е екипът, извършващ прегледа да включва хора от различни специалности – като са задължителни поне един експерт в приложната област на системата и 1 краен ползвател.
- Прототипиране – създава се прототип, който да демонстрира изискванията и да помогне на ползвателите да открият проблемите. Тази техника си заслужава само в случай, че при извличането на изискванията вече е разработен прототип. Прототипът за валидиране трябва да е пълен, ефикасен, надежден и дуракоустойчив.
- Валидиране на модел на системата – етап от процеса на валидиране. Цели да провери дали всеки модел на системата е логичен и цялостен, дали отделните модели са съвместими и дали те представят реалните изисквания на ползвателите на системата. Една от техниките за валидиране на модел е перефразиране на модел- изискванията да бъдат систематично преведени на описания на естествен език. Има 3 вида перефразиране:
    - Шаблони – от моделите се генерира перефразирано описание, позволяващо да се открие липсващата информация.
    - „Автоматизирани“ шаблони – обикновено се използват Case tool-ове, които докладват за проблеми в модела.
    - Математическо доказателство – използва се при използването на формални спецификации и модели.
- Тестване на изискванията – всяко изискване трябва да бъде тестируемо и невъзможността да се създаде тест говори за липсваща или неясна информация в описанието на изискванията. Всяко функционално изискване трябва да бъде анализирано с подходящ тест.

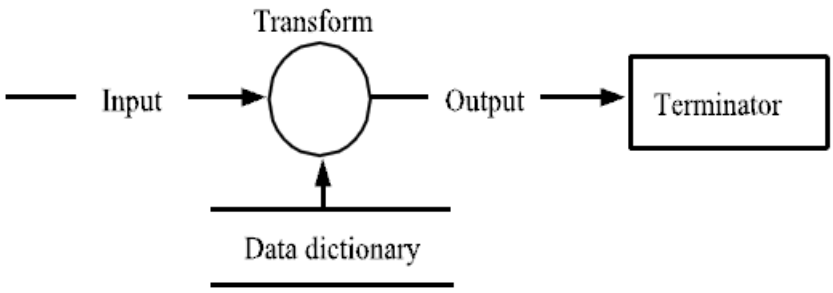
5) Основни методи за моделиране

Процесът на инжинеринг на изискванията обикновено е съпроводен от метод за моделиране. Тези методи представляват систематични начини за изготвяне на системни модели.

Data-flow Модели

Data-Flow моделите се базират на идеята, че системите могат да бъдат моделирани като множество от взаимодействащи си функции. Използват data-flow диаграми (DFD) за графично представяне на външните същности (entities), процеси, data-flow и data stores. DFD имат важна роля при определянето на данните, достъпни в даден компонент, чрез което помагат за разбирането на това какво трябва да прави компонентът и как ще завършва задачите си. DFD показват потока на данни между множество компоненти, реализиращи функционалността на системата. Актьорите не се включват в DFD. Последователността на действията може да бъде заключена от последователността от activity boxes.

Нотацията при DFD е следната:



- стрелките прдставят данните
- кутиите са процеси

Data-flow подходът се представя от метода за структурен анализ. При него има 2 стратегии:

- стрaгетия на ДеМарко - възприема top-down подход, при който аналистът се опива да съпостави текущата физическа система на текущия логически data-flow модел.
- модерен структурен анализ - прави разграничение между нуждите на потребителя и изискванията за външното поведение на системата, задоволяващи тези нужди.

Семантичен модел

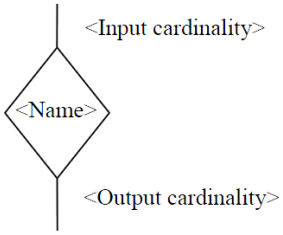
Използва се за описване на логическата стуктура на данните в системата. Идентифицира същоностите (entities) в базата данни, атрибутите им и връзките между тях. Използва графични нотации. Подходите към семантичното моделиране на дните включват Entity-relationship модел, RM/T, SDM.

Нотацията за семантичните модели надани е следната:

- същност (entity)

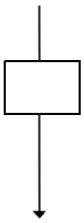


- релация между същности



A relation between entities

- релция за наследяване



An inheritance relation

Обектно-ориентираният модел

Обектно-ориентираният модел е базиран на идеята, че системите могат да бъдат моделирани като множество от взаимодействащи си обекти. Основния подход при обекно-ориентираната методология е да се разработи обектен модел, описващ приложната област. Основната цел е моделирането на приложната област, а не проектирането на имплементацията. Следните основни правила важат за обектните модели на изискванията:

- всички същности от реалния свят, важни за разбирането на приложната област, трябва да бъдат включени
- всички методи и атрибути, важни за разбирането на приложната област, трябва да бъдат включени

- обектите, атрибутите и методите, които са важни за имплементацията, НЕ трябва да бъдат включени.

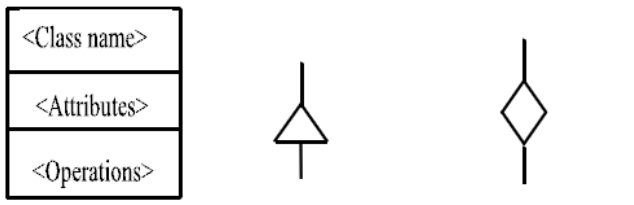
Обектните модели представят същности и връзките между тях.
При този подход се моделират обектите, класовете и операциите, които те предоставят. Основните понятия са:

- обект – идентифицират се при анализа на приложната област. Обектите са нещо реално или абстрактно, за което се съхраняват данни и операции, които манипулират тези данни. Могат да се състоят от други обекти.
- клас – имплементация на тип обекти, които съдържат общи атрибути и операции. Обектите са инстанции на класовете.
- Операции и методи – използват се за четене и манипулация на данните на обект. Реферират единствено структурите от данни на обектен тип. За достъп до данните на друг обект, обектите трябва да изпратят съобщние до него. Методите описват начин, по който операциите са реализирани в софтуера.
- Енкапсулация – групирането на данни и операциите, манипулиращи тези данни. Предодвратява неоторизирания достъп до данните на обекта и скрива детайлите за изпълнението на операцията.
- Наследяване
- Съобщения – обектите комуникират помежду си чрез изпращането на съобщения. Съобщенията се състоят от името на получаващия обет, операцията, който да бъде изпълнене, параметри. Получаването на съобщение предизвиква извикването на операция и изпълнението на подходящия метод върху получаващия обект.

При използването на обектно-ориентирания модел се изпълняват следните стъпки:

- Идентифициране на основните (core) обекти.
- Конструиране на обекните структури, дефиниращи асоциациите между класовете.
- Дефиниране на атрибутите на всеки обект.
- Определяне на операциите за всеки обект. – могат да се идентифицират чрез сценарии и use cases.
- Дефиниране на съобщенията, които се предават между обектите.

Нотацията, която се използва при обектно-ориентирания модел е:



(i) Class (ii) Generalisation (iii) Aggregation

Всяка кутия представя вид обект. Името, атрибутите и методите на обекта са изброени в кутията. Стрелка между 2 обекта представя връзка между тях. Стрелките могат да са именувани с името на асоциацията и във всеки край може да се задава число, указващо колко различни асоциации от този тип са позволени. Има 3 типа връзки:

- Наследяване – обектът от долу е специален случай на този отгоре. Представя “is-a” връзка.
- Агрегация – задава, че единият обект е компонент на другия. Представя “part-a” връзка.
- Асоциация – един от обектите асоцииран с другия.

Формални методи

Формалните методи се базират на математически принципи и целят да постигнат голяма убеденост, че системата ще отговаря на спецификацията си. Състоят се от:

- синтаксис – дефинира нотацията, чрез която се представя спецификацията.
- семантика - помага за дафинирането на обектите, които ще бъдат използвани за описанието на системата.
- релации – дефинират правилата, по които се посочва кои обекти отговарят на спецификацията.

Формалната спецификация използва формално дефиниран модел, за да описва поведението на софтуера. Описанията обикновено попадат в 3 категории:

- precondition – твърдение, което трябва да е врно преди изпълнението на функция.
- post-condition - твърдение, което трябва да е врно след изпълнението на функция.
- invariant – условие, което винаги е вярно преди и след изпълнението на функция.

Формалните методи се делят на 2 категории:

- Нотации, базирани на модели (model-based notations) – Z, VDM.
- Нотации, базирани на process algebras – LOTOS, CSP, CCS.

Обикновено се прилагат в проекти, в които сигурността и безопасността са критични.
Предимствата на формалните методи са, че те премахват двусмислеността, насърчават по-голяма точност в началните етапи на разработка на проекта и позволяват верифицирането на коректността, неточността и противоречивостта на спецификацията. Недостатъците са, че чрез тях е трудно да се представят поведенческите аспекти на проблема, не адресират проблема за дефинирането на изисквания, не е поддържат от достатъчно tool-ове.

18. Инженеринг на изискванията като част от системния и софтуерен инженеринг. Представяне на техники, прилагани на отделните етапи от инженеринга на софтуерните изисквания. Основни методи за моделиране.
Изложението по въпроса трябва да включва следните по-съществени елементи:
1) Цел и задачи на инженеринга на софтуерните изисквания.
2) Параметри на процеса – видове изисквания, класификация, потребители.
3) Етапи (дейности) на инженеринга на изискванията.
4) Техники за извличане, анализ и валидиране на изискванията.
5) Основни методи за моделиране – Data-flow модели, семантични модели, обектно-ориентирани модели, формални модели.

1. Инженеринга на софтуерните изисквания е свързан с идентифицирането на целите на софтуерната система и контекста, в който тя ще бъде използвана

Инженеринга на изискванията е процеса на разбирането и дефинирането на услугите, които ще предоставя системата и идентифицирането на ограниченията към системата. Това е ключова фаза от софтуерния процес, тъй като грешки на този етап водят до грешки в дизайна и имплементирането на системата, което води до загуба на време и пари. Изходния продукт от този етап е документ със спецификацията на системата. Целта на инженеринга на изискванията е да се създаде документ с изискванията към системата. Този етап от разработката на една софтуерна сиситема трябва да идентифицира проблема – кои проблеми трябва да бъдат решени, къде е проблема (разбирането на контекста), на кой е проблема (идентифицирането на заинтересованите лица), защо е нужно разрешаването му (идентифицирането на целите на заинтересованите лица), как софтуерната система би помогнала (събиране на различни сценарии), кога трябва да бъде разрешен (идентифицирането на ограничения за разработването), какво би попречило разрешаването му (идентифицирането на рисковете). Целия процес по инженеринга на изискванията се състои от четири подпроцеса- анализ/проучване за осъществимост на системата (feasibility study), откриване на изискванията – извличане и анализ(elicitation and analysis), преобразуване на изискванията в някаква стандартна форма (specification) и проверка за това дали изискванията наистина отговарят на това, което потребителя очаква от системата (validation). Влияние върху процеса на инженеринга на изискванията имат всички заинтересовани лица: клиентите – тези, които плащат за да бъде разработена системата и тези, които закупуват системата след като тя бъде разработена, потребителите – ще използват системата, експертите от приложната област, които са запознати с проблемите, които трябва да бъдат решени от системата, маркетинг експерти, които са проучили нуждите на клиентите, адвокати и одитори и софтуерни разработчици.
Какви са възможните източници на изисквания: хората – всички заинтересовани лица, системи – с какви други системи ще си взаимодейства нашата система и от документи – маркетингови проучвания, стандарти, анализи за приложната среда и други.

2. Изискванията трябва да бъдат коректни (correct), консистентни (consistent), ясни (unambiguous), пълни (complete), изпълними (feasible), приложими (relevant), тестериуеми (testable) и с възможност за проследяване (traceable).

В зависимост от нивото на описание на изискванията те биват, потребителски изисквания и системни изисквания.

Потребителските изисквания на софтуерната сиситема трябва да описват както функционалните така и нефункционалните изисквания и да бъдат разбираеми за потребителите на сиситемата. Те трябва да описват само външното поведение на сиситемата без да се включва информация за дизайна. Не трябва да се използва софтуерен жаргон, нотации или да се включва информация за имппламентацията на системата. Потребителските изисквания трябва да са на достъперн език, представени чрез таблици или диаграми. Могат да възникнат следните проблеми при описанието на изискванията: няколко различни изисквания могат да бъдат описани като едно изискване, липса на достатъчно яснота по време на описването на изискванията, различните видове изисквания не са описани достатъчно ясно и не могат да се разграничат.

Потребителските изисквания са с по-високо ниво на абстракция, докато системните са детайлно описание на това какво ще прави системата.Потребителските изисквания са твърдения на достъпен език и диаграми за това какви услуги ще предоставя системата и ограниченията, при които трябва да работи.Системните изисквания са детайлни и описват точната функционалност и ограничения на системата.

Системните изисквания са разширена версия на потребителските, използват се от софтуерните инженери като отправна точка за правенето на дизайна на системата. Те са по-детайлни и изясняват как потребителските изисквания ще бъдат внедрени в системата. Системните изисквания също трябва да бъдат написани на достъпен език, въпреки че това може да е трудно.

Друг начина за класифицирането на изискванията е на функционални и нефункционални и такива произлизащи от приложната област (бизнеса).Функционалните описват услугите и функциите, които системата трябва да предоставя на потребителите. Какво системата ще прави при определени условия или входни и изходни данни. Функционалните изисквания трябва да бъдат пълни и консистентни. Пълни означава, че всички услуги, които потребителя очаква от системата трябва да са описани. Консистентни означава, че изискванията не трябва да бъдат дефинирани противоречиво Нефункционалните включват ограничения към системата или процеса на разработването и.

Нефункционалните изисквания са свързани с качеството на системата. Могат да се разделят в следните категории. Изисквания произтичащи от продукта - описват

поведението на продукта. Колко бързо системата ще изпълнява определено действие, колко памет е необходима, колко е надеждна системата, portability, usability
Организационни - произлизат от политиките или процедурите в организацията на потребителите или разработчиците. Стандарти, които трябва да бъдат спазени, език за програмиране, който трябва да се използва, дизайн, кога продукта и документацията трябва да бъдат завършени. Външни - произтичат от всички фактори, които са външни за системата или за процеса по разработването и. Например как системата ще си взаимодейства с вече съществуващите системи на клиента, етични изисквания...

Нефункционалните изисквания трябва да се описват с метрики, така че да могат да се тестват Нефункционалните изисквания често са в конфликт или зависят от функционалните.
Бизнес изискванията могат да бъдат както функционални така и нефункционални и се извлича от приложната област към, която ще принадлежи системата. Описва защо има нужда от тази система или какво иска да постигне чрез системата. Този тип изисквания идват от спонсорите на проекта или от мениджърите и са изисквания на високо ниво.

Устойчивост на изискванията (stability)
- постоянни изисквания – такива, които не се променят по време на разработката на системата
- непостоянни (изменчиви) изисквания – изисквания, които се променят по врене на разработката или, когато системата влезе в употреба

Приоритизиране на изискванията:
- изисквания, които задължително трябва да бъдат изпълнени
- изисквания, които са желателни
- изисквания, на които е възможно да отговаря системата, но могат да бъдат пренебрегнати

3.
Основните етапи при инженеринг на изисквания са: извличане на изискванията, анализ, валидиране и управление. **Извличането** на изискванията става с помощта на клиентите, открива се приложната област, услугите, които ще предоставя системата и какви са ограниченията. Дейностите свързани с извличането са: откриването на приложаната среда – къде системата ще бъде използвана, разбиране на проблема (нуждите), за които системата ще бъде разработена, разбиране на бизнес средата, в която системата ще бъде използвана и разбиране на нуждите и ограниченията на заинтересованите лица. По врема на тази фаза възникват проблеми породени от факта, че заинтересованите лица не винаги знаят какво искат, изразяват се със собствени термини, по време на разработването се появяват нови заинтересовани лица, организационни и политически различия. Изходния продукт от тази дейност е чернова с изискванията.
Фазата на **анализа** на изискванията проверява дали всички изисквания са нужни и необходими за разработването на сиситемата. Прави се проверка дали изискванията са

пълни и консистентни – изискванията трябва да не бъдат противоречиви и да бъде описано всичко за сиситемата. Проверява се дали са изпълними спрямо бюджета и графика за разработването на системата. Обсъждат се със заинтересованите лица и се постига съгласие. Изискванията се приоритизират, като се взима предвид кои от тях са с критична важност за системата. По време на анализа е неободимо да се отговори на следните въпроси: дали има изисквания за предварителен дизайн иии информацията за внедряването, дали дадено изискване може да бъде разделено на няколко подизисквания, дали всички описани изисквания наистина са необходими, дали изискванията не са двусмислени – могат да се разбират по различен начин от различните заинтересовни лица, дали изискванията са реалиситчни, дали изискванията са тестеруеми

Валидацията на изискванията удостоверява, че документът, описващ изискванията е, приемливо описание на системата, която ще бъде реализирана. Документът на изискванията се проверява за пълнота и последователност, съответствие със стандартите, конфликт в изискванията, неяснота на изискванията, технически грешки. Валидирането работи с вече подготвени и съгласувани изисквания. Входна инфрмация за процеса на валидирането е документ на изискванията,знание за организацията и организационни стандарти – трябва да бъде проверено спазването на локални стандарти. Документът с изискванията трябва да е цялостна завършена версия на документа. Изготвянето и форматът на документа е съобразен със стандартите на организацията.Знанието за организацията включва – използвана терминология, умения на работещите, култура и т.н. Изходната информация от процеса на валидирането е списък на откритите проблеми в описанието на изискванията – откритите проблеми се класифицират според типа им и списък на действията, които трябва да се извършат за коригирането на изискванията.

4) Техники за извличане, анализ и валидиране на изискванията
Техники за извличане и анализ на изискванията: интервюта, сценарии, FAST, Soft System Methods, наблюдение, преизползване на изискванията
Формални или неформални *интервюта* със заинтересованите лица е една от техниките за извличане на изискванията. Тя се използва също и при анализа. Екипа по инженеринга на изискванията задава въпроси на заинтересованите лица за системата, която трябва да се разработи. Изискванията се извличат от отговорите на поставените въпроси. Интервютата биват затворени, отворени или смесен тип. При затворените интервюта заинтересованите лица отговарят на набор от предварително подготвени въпроси, докато при отворените интервюта се задавт въпроси, които не са предварително подготвени, а възникват по време на интервюто в зависиост от дадените отговори. В повечето случаи се използва смесения тип интервюта. Отговора на един въпрос води до друг въпрос и така до пълното изясняване на изискванията към системата. Интервютата са подходяща техника за придобиване на цялостна информация за системата. За да бъдат полезни заинтересованите лица трябва да бъдат открити и да им се даде отправна точка за дискусия.
Сценарии:

Сценариите предтавляват релален пример за това как системата ще бъде използвана. Те трябва да съдържат описание на това което системата и потребителите очакват, когато системата се стартира; описание на нормалния поток от събития; описание на това какво може да се объркa (повреди); информация за други конкурентни дейности; описание на състоянието на системата, когато сценария свършва

Use Case:

Това е техника базирана на сценариите но се използва обектен подход за описването и.

- Use-cases are a scenario based technique in the UML which identify the actors in an interaction and which describe the interaction itself.
- A set of use cases should describe all possible interactions with the system.
- Sequence diagrams may be used to add detail to use-cases by showing the sequence of event processing in the system.

Facilitated Application Specification Technique

FAST

Срещите се състоят на неутрална среда, има правила за провеждане на срещтата, могат да бъдат формални и неформални…

- Meeting at neutral site
- Rules for preparation & participation
- Agenda – formal + informal
- Facilitator controls meeting
- Definition mechanism
- Identify problem, propose solution, negotiate approaches, specify set of solution requirements

Soft System Methods

- These produce informal models of a socio-technical system. They consider the system, the people and the organisation
- Not techniques for detailed requirements elicitation. Rather, they are ways of understanding a problem and its organisational context
- Software Systems Methodology (SSM) is probably the best known of these methods
- The essence of SSM is its recognition that systems are embedded in a wider human and organisational context

Stages of SSM

- Problem situation assessment
- Problem situation description
- Abstract system definition from selected viewpoints
- Conceptual system modelling
- Model/real-world comparison
- Change identification
- Recommendations for action

Наблюдение и социален анализ: Хората често изпитват затруднения да опишат какво правят в своята работа, защото това се е превърнало за тях в нещо естествено. Затова добър начин да разбереш с какво е свързана работата им е наблюдението по време на работата. Това помага да се разбере работния процес и действията, които извършват хората и да се изгради представа за техните нужди.

Преизползване на изискванията:
Взимат се изисквания, които са били открити за една система и се използват за друга. Преизползването на изискванията спестява време и усилия, тъй като изискванията, които се използват вече са били валидирани и анализирани за предишната система.
Прототипиране: изгражда се прототип, който да извлече или валидира изискванията
Има два вида прототипа:
- throw away помага за извличането на изискванията, изискванията които се включват в прототипа са тези, които са най-трудни за разбиране. Отразява най-неясните изисквания към системата.
- evolutionary prototyping – доставя на потребителите работеща система, в него се включват изискванията, които са лесни за разбиране и могат да предоставят на крайния потребител желаната от него функционалност.
Прототипирането може да доведе до загуба на време, ако системата не е сложна и изискванията са достатъчно ясни. Могат да бъдат използвани много ресурси (време, пари), а прототипът да не задоволява нуждите на клиентите.

Техники за валидиране: преразглеждане/преглед/рецензиране на изискванията. Формира се специална група, която да се запознае с изискванията и да ги анализира; формулира забелязани проблеми; дискутира възможните действия за справяне с проблемите.
Дейностите сврзани с прегледа на изискванията са: **Планиране -** Определя се екипа, ръководител и протоколчик; Избира се време и място за срещите; **Разпространение** на документите за преглед до участниците в екипа; **Подготовка** за прегледа - Всеки рецензент да прочете документа за изискванията и да набележи открити конфликти, пропуски, непоследователности, отклонения от стандартите, др. проблеми; **Среща по прегледа** - Дискутират се коментарии и проблеми на отделни участници от екипа и се дефинира набора от възможни действия за справяне с проблемите; **Проследяване на действията** - Ръководителят на екипа проверява дали формулираните действия са изпълнени; **Ревизиране на документа -** Документът на изискванията се редактира, за да отрази съгласуваните действия. На този етап той може да бъде предаден като окончарелен вариант или върнат за нов преглед.
//Действия за решаване на всеки отделен проблем:
Изясняване на изискването, защото То може да е лошо представено или да има случайно изтървана информация. Презаписване на изискването.
Липсваща информация Отговорност на инженерите по изискванията е да открият тази информация от съответен източник – ползвател или др.
Конфликт в изискванията Ако съществува значим конфликт в изискванията, то ползвателите трябва да преговарят, за да достигнат до съгласие.

Нерелистични изисквания Изискването не би могло да бъде реализирано, защото има технологични и/или други ограничение върху системата. Необходимо е ползвателите да се консултират, как да направят изискването по- реалистично. //

Участници в екипа – формирането на екипа е от съществено значение за качествения преглед

• Част от рецензентите трябва да включват ползватели с различни специалности, за да се съберат различни умения и знания;
• Участвайки в процеса на прегледа ползвателите от различни специалности могат да разберат по-добре необходимостта от промяна на изискванията;
• В екипа трябва задължително да се включи:
поне един *експерт от специфичната област* на създаваната система, както и понеедин *краен ползвател*.
Брой на експертите, формиращи екипа;
Организиране на работата на екипа.

Прототипиране
Прототипите, предназначени за валидиране на изискванията, имат за цел да **демонстрират изискванията** и **да помогнат на ползвателите да открият проблемите**. Прототипите за валидиране трябва да бъдат **пълни, ефекасни** и **робастни**. Те трябва да могат да се ползват по същия начин, по който ще се използват в исканата система. Документация и ръководство за ползвателя трябва да бъдат предоставено.Валидирането може да започне с непълен прототип, но задължително системата трябва да бъде допълнена и довъроена по време на валидирането.Надеждността на прототипа е задължителна.Техниката на прототипиране заслужава да се прилага ***само*** вслучай, че вече е разработен прототип за целите на извличане на изискванията!

Действия при прототипирането: успоредни процеси
Избор на подходящ екип за тестване на прототипа Най-подходящи са тези ползватели, които имат умерен опит и които са “отворени” към използването на нови системи. Крайни ползватели с различен характер на извършваната дейност трябва да участват, за да се изпробват/покрият различни функционалности.
Разработване на тестови сценарии Внимателното планиране предполага формулиране на пълен набор от тестови сценарии, които осигуряват широко покритие на изискванията. (End-users shouldn’t just play around with the system as this may never exercise critical system features).
Изпълнение на сценариите При изпълнение на сценариите ползвателите е добре да работят самостоятелно, като по този начин симулират и реалната ситуация на използването на системата.
Документиране на проблемите Електронен или хартиен документ с откритите проблеми е препоръчителен

5. Методите за инжинеринга на изискванията могат да бъдат категоризирани като формални, полуформални и неформални.
Полуформалните и неформалните използват обикновен (достъпен) език, таблици и диаграми. Включват структурни и обектно ориентирани анализи. Формалните методи са базирани на математически синтаксис и семантика. Такива методи са Z, B, VDM, LOTOS.
Формалните методи:
Provide a means for achieving a high degree of confidence that a system will conform to its specification; Do not absolute guarantee of correctness;
Have little directly to offer to the problems of managing software projects. However, benefits can be gained from gaining a clear understanding of the task at an early stage.
Формалните методи се състоят от синтаксис, семантика и връзка между тях. Синтаксиса показва специфичната нотация, която се използва при метода(that defines the specific notation with which the specification is represented – often based on a syntax that is derived from standard set theory notation and predicate calculus). Семантиката определя как се представят изискванията (that help to define a "universe of objects" that will be used to describe the system – how the language represents system requirements). Релацията (*Relations* which define the rules that indicate which objects properly satisfy the specification).
Формалните методи не са широко разпространени, тъй като нотацията им е трудна за разбиране, трудно с тяхна помощ се описват определени аспекти от изискванията, липсват инструменти, които да правят употребата им по-лесна. Предимства на формалните методи са, че тяхната употреба намалява двусмислеността, тъй като се използват математически формализъм, което прави възможно проверката за пълнота, коректност и консистентност. При използването на тези методи се изисква по-голяма точност и взискателност още в ранната фаза от инженеринга на изискванията. Формалните методи могат да се разделят в две категории - Model-based notations – Z and Vienna Development Method (VDM) и Process algebras -based notations - *Communicating Sequential Processes (CSP), CCS and LOTOS.*

Обектно - ориентирания подход е базиран на идеята, че системата може да бъде моделирана като набор от взаимодействащи си обекти.

Data – flow model е базиран на идеята, че системата може да бъде представена като набор от взаимодействащи си функции.